CABINET CHEAT SHEET

Power Glove Vision Quick Reference

Current cabinet addresses, services, controls, and maintenance commands.

2026 EDITION / IAIN BENNETT /

Use this guide to install, pair, check, and operate your PowerGlove Vision system. The **PowerGlove Vision Controller** is the project's camera and recognition device, built on an Arduino UNO Q. Replace `UNO-Q-NAME.local` with your PowerGlove Vision Controller hostname and `RETROPIE-NAME.local` with your Raspberry Pi hostname. Each command section identifies the machine on which to run it. Keep passwords and pairing tokens out of this document.

Your installation

Item	Value
PowerGlove Vision Controller network address	`UNO-Q-NAME.local`
RetroPie network address	`RETROPIE-NAME.local`
PowerGlove Vision Controller App Lab application	PowerGlove Vision
PowerGlove Vision Controller application directory	`/home/arduino/ArduinoApps/powerglove-vision`
Camera	UVC-compatible USB camera; select `auto` in Setup
Startup profile	Choose in Setup

Prefer `.local` names in bookmarks and settings. If a name does not resolve, check the device's current IP address in your router and use that address temporarily. A router reservation prevents the fallback address from changing.

Browser URLs

Open these pages on a computer or phone connected to the same trusted network.

Page	Address
Dashboard: camera and controller output	Open Dashboard
Play: Rock Paper Scissors	Open Play
Learn: practice and tune gestures	Open Learn
Games: edit game mappings	Open Games
Help: manuals and live cabinet reference	Open Help
Setup: connection and startup settings	Open Setup
Secure Setup: pairing	Open secure Setup
Status: diagnostic readings	Open status
Camera stream	Open camera stream
Project repository	PowerGlove Vision on GitHub

The links above contain example hostnames. Replace them in the browser's address bar. The live **Help > This cabinet** page builds links using the PowerGlove Vision Controller address you used to open it.

Install and deploy over Wi-Fi

For a first installation, follow these sections in order, then pair the devices and complete the Installation Guide's gameplay checks. All flags are explained in the command reference.

Download and package on your computer

1. Install Arduino App Lab on your development computer. Run `command -v git python3 bash rsync zip` and install any missing tools.
2. In a macOS or Linux terminal, choose your projects folder and run the commands below. They download the current `main` branch and build its PowerGlove Vision Controller installation ZIP.
3. Confirm that verification reports **App Lab installation ZIP verified**.

```sh
SH
git clone --branch main https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
scripts/build-app-lab-package.sh
python3 scripts/verify-app-lab-package.py
```

Prepare the PowerGlove Vision Controller

1. Connect the PowerGlove Vision Controller by USB and complete its setup in App Lab. Join the same network as RetroPie and record the board's hostname.
2. Import `output/app-lab/PowerGlove-Vision-Uno-Q.zip` from your computer's checkout. Open **PowerGlove Vision** and select **Run** to transfer and start the app and matrix sketch.
3. Connect the camera through the powered USB hub. Follow the Installation Guide if you need help with the initial board setup.

Open a terminal on the PowerGlove Vision Controller, or connect from your computer:

```sh
SH
ssh arduino@UNO-Q-NAME.local
```

Run these commands **on the PowerGlove Vision Controller** after importing and running the app once:

```sh
SH
cd /home/arduino/ArduinoApps/powerglove-vision
sudo python3 scripts/setup-machine.py uno-q
```

This installs host support for local names, shutdown, and guarded USB-camera recovery, sets the app to start at boot, and restarts it. Review every **FAIL** or **ACTION** result. The installer requires the application directory shown above. Run `exit` after setup to leave the PowerGlove Vision Controller terminal. Check Dashboard, Play, and Learn before pairing.

Install the Raspberry Pi receiver before pairing

The Raspberry Pi needs the receiver, pairing command, game registry, and launch hooks before you can pair it with the PowerGlove Vision Controller. Run the following in a terminal **on the Raspberry Pi running RetroPie**. You can use a local terminal or SSH with your RetroPie account.

For a new installation, download the same `main` branch used on the PowerGlove Vision Controller:

```sh
sudo apt update
sudo apt install -y git
cd ~
git clone --branch main https://github.com/mathan416/PowerGlove-Vision.git
cd PowerGlove-Vision
```

If you already have a checkout, open that directory instead of cloning again. Then install the RetroPie components, substituting your PowerGlove Vision Controller hostname:

```sh
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

The installer preserves existing tokens and settings, installs the receiver and pairing commands under `/opt/powerglove/bin/`, and adds the game-launch hooks. It also installs the controller mapping and the 45-second startup timer. An **ACTION** result asking you to pair or verify gameplay is expected on first installation. Correct any **FAIL** result before continuing to pairing.

If a registered Super Glove Ball ROM is present, the installer offers to build the optional `lr-nestopia-powerglove` core from its pinned Nestopia source. It installs under a separate name and leaves stock Nestopia untouched. The ROM's saved emulator remains FCEUmm until you explicitly choose the native core from RetroPie's per-ROM launch menu. Declining the optional build leaves the complete FCEUmm fallback available.

For an existing installation, `--peer` does not replace the saved PowerGlove Vision Controller address. If that address has changed, update `/etc/powerglove/launcher.json` on RetroPie.

Update the PowerGlove Vision Controller from your computer

Complete the SSH key setup first. From the full project checkout **on your development computer**, verify access and deploy:

```sh
ssh -o BatchMode=yes arduino@UNO-Q-NAME.local hostname
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

The deployment preserves the PowerGlove Vision Controller's private `data/` directory and restarts the application. It updates the PowerGlove Vision Controller only. To update RetroPie, update its source checkout and rerun the RetroPie installer above; it preserves local settings.

The PowerGlove Vision Controller installer includes the shutdown and camera-recovery helpers. To update or repair them separately, run this from your development computer's project checkout:

```sh
scripts/install-uno-q-shutdown-helper.sh arduino@UNO-Q-NAME.local
```

To install or repair only camera recovery without touching shutdown support:

```sh
scripts/install-uno-q-camera-recovery-helper.sh arduino@UNO-Q-NAME.local
```

The camera does not need to be connected during installation. The first healthy camera sighting enrolls the one UVC camera and its actual parent hub. Moving the camera to another hub updates the association automatically the next time vision sees it. Until that first sighting, recovery intentionally has no hub to reset.

The terminal prompts for the PowerGlove Vision Controller account password if needed. The helper requests a Linux halt; the tested board restarts afterward. See the shutdown limitation below.

Pair your RetroPie

Complete both machine installations above, then use the one-time-code method:

1. On RetroPie, run `sudo /opt/powerglove/bin/powerglove-pair` and leave it running. Its code expires after two minutes.
2. In your browser, open `https://UNO-Q-NAME.local:8443/setup` using your PowerGlove Vision Controller's actual hostname.
3. Enter your RetroPie hostname and the 20-character code printed by the pairing command.
4. Select **Prepare one-time code**. Compare the matrix `ID` with the beginning of the browser certificate's SHA-256 fingerprint.
5. If they match, select the certificate confirmation checkbox, enter the six-digit PIN displayed after `PN` on the matrix, and select **Complete pairing**.
6. On RetroPie, run `sudo systemctl status powerglove-receiver.service` and confirm that the receiver is active.

Password pairing is available on the same secure page when RetroPie accepts SSH password login. Select **Prepare password pairing**, complete the same physical certificate check, and then enter your RetroPie account password. The password is used for one SSH operation and is not stored.

Quick health checks

From your computer, open the status URL above or run:

```sh
curl -sS http://UNO-Q-NAME.local:8088/status
```

Once you have selected an active profile and completed calibration, the status readings should show the following while your hand is visible:

```json
{
  "camera_available": true,
  "worker_running": true,
  "detected": true,
  "calibrated": true
}
```

With **Gestures off**, an inactive camera is normal. Select a profile on Dashboard or open Learn to check tracking.

Run these checks **on RetroPie**:

```sh
SH
sudo systemctl status powerglove-receiver.service
sudo systemctl status powerglove-receiver.timer
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
grep -A8 -B2 'PowerGlove Vision' /proc/bus/input/devices
```

The virtual controller appears after the first authenticated packet. Select **Start controller** on Dashboard when you are ready to send input.

Run the local software tests

This is a developer check. Run it from the **root of the full Git checkout**, where the `src/` and `tests/` directories are present:

```sh
SH
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

The command is correct for macOS and Linux. `PYTHONPATH=src` lets the tests import the local source without installing the package. The tests do not require a camera, MediaPipe, or either physical device, but some open temporary local network listeners. The App Lab ZIP and the installed receiver files do not include the full test suite. Successful completion ends with `OK`; these tests do not replace a check of the controls in a running game.

Camera troubleshooting

Connect the camera to the **PowerGlove Vision Controller** through a powered USB hub. A camera attached to your computer is not available to the PowerGlove Vision Controller application.

Open a terminal on the PowerGlove Vision Controller, using SSH if necessary:

```sh
SH
ssh arduino@UNO-Q-NAME.local
```

Run the following **on the PowerGlove Vision Controller** to see each Linux video device and its name:

```sh
SH
for device in /sys/class/video4linux/video*; do
  [ -r "$device/name" ] || continue
  printf '%s: ' "/dev/${device##*/}"
  cat "$device/name"
done
```

Entries named `qcom-venus-encoder` or `qcom-venus-decoder` are the board's internal video codecs, not the webcam. Look for an additional device whose name matches your USB camera. If only codec entries appear, or no entries appear, Linux has not exposed a webcam video device. Check hub power, reconnect the camera, and run the command again.

You can also check for persistent USB-camera links:

```sh
ls -l /dev/v4l/by-id/
```

That directory may be absent on some systems; its absence alone does not prove that the camera is missing. Use the device-name listing above as well.

Return to Dashboard, select an active profile, and watch for the camera view. The app retries camera initialization automatically. Keep **Camera** set to `auto` unless you have identified a specific capture device to select.

Place the camera before calibrating

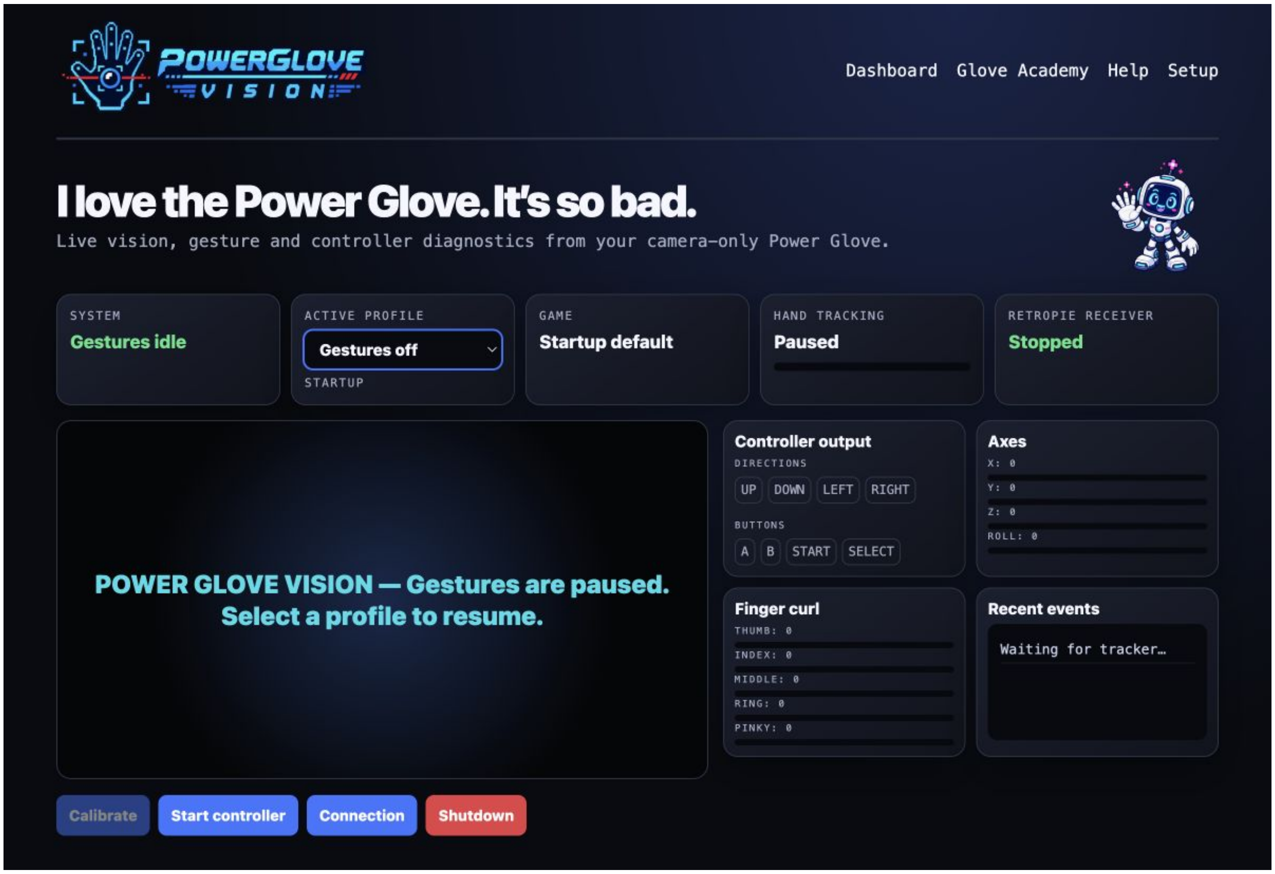
1. Put the camera in its normal cabinet position before calibration.
2. Stand or sit at your normal playing distance. Keep your comfortable center and the full area you intend to reach inside the camera view, with room at every edge.
3. Hold a relaxed open hand at that center and select **Calibrate**. Direction thresholds are shared across games and automatically rise above measured resting-hand jitter; separate left, right, up, and down calibration is not normally needed.
4. After checking the live view, close Dashboard or the direct camera stream while playing. Tracking and controller delivery continue, while closing the 5 fps preview reduces avoidable PowerGlove Vision Controller work and game stutter.

Recalibrate after moving the camera, changing your playing distance, or changing your normal center. Returning to the same position produces a similar reference, although normal camera variation means the saved numbers will not be identical.

Website screenshots

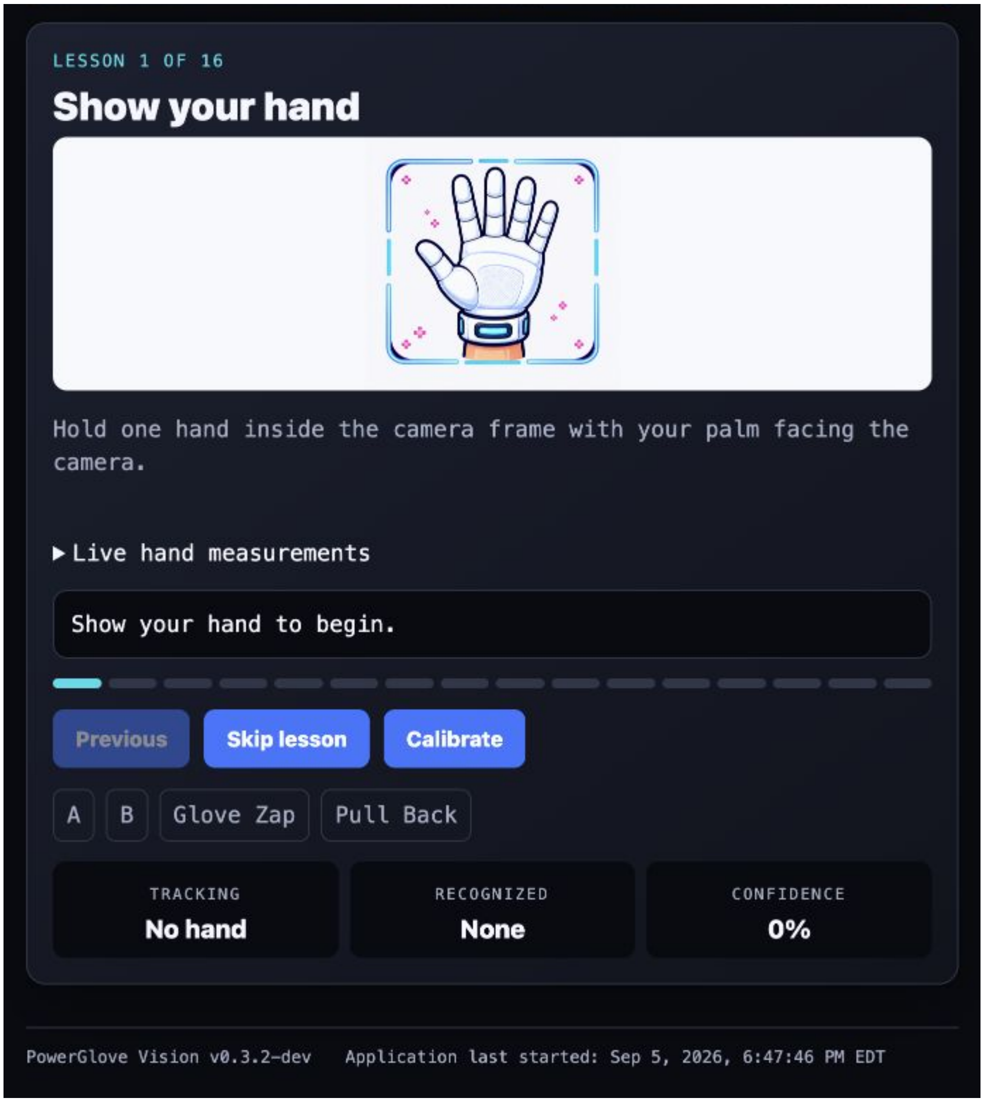
These are reference screenshots, not live views. Open the browser URLs above to see your own camera and controller status. Camera imagery is blurred for privacy. These screenshots were refreshed on September 4, 2026.

Dashboard



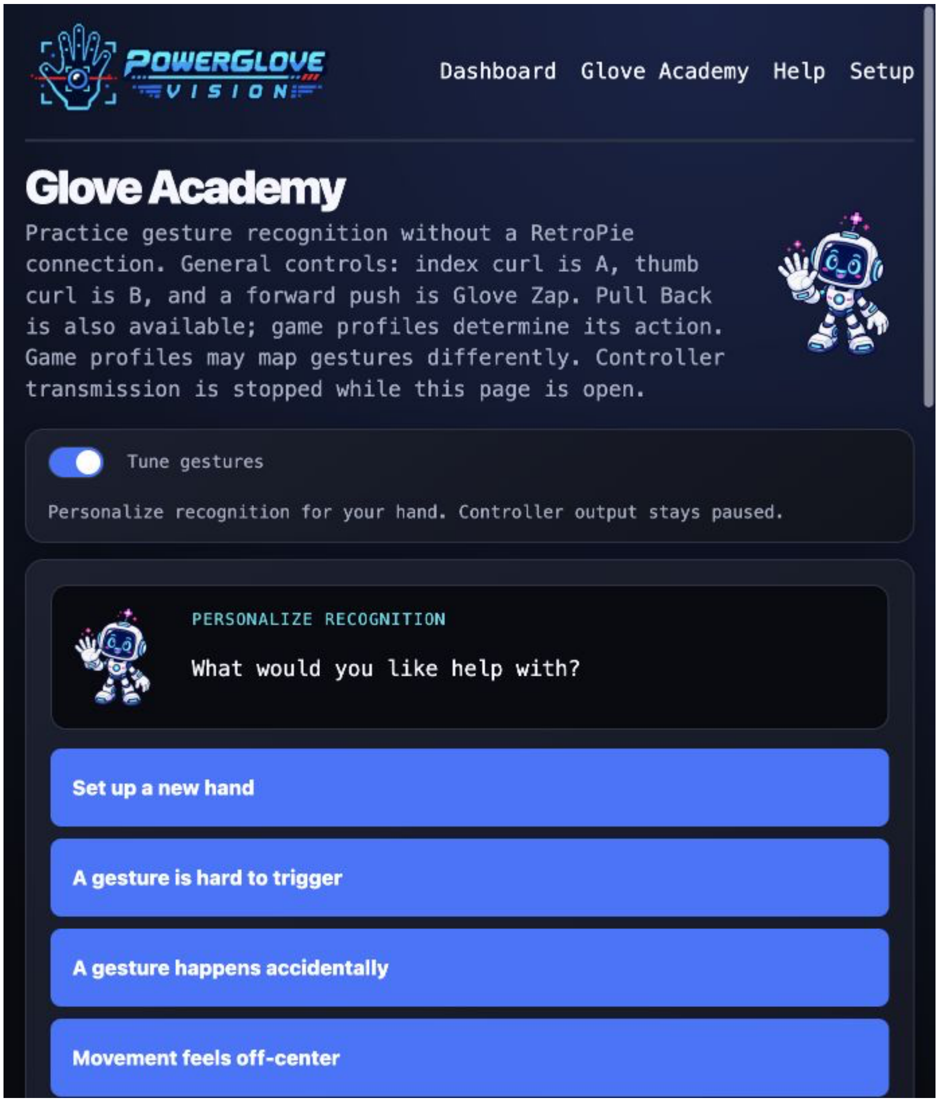
Dashboard showing Gestures off and controller diagnostics

Glove Academy



Glove Academy practice lesson with its live camera area excluded for privacy

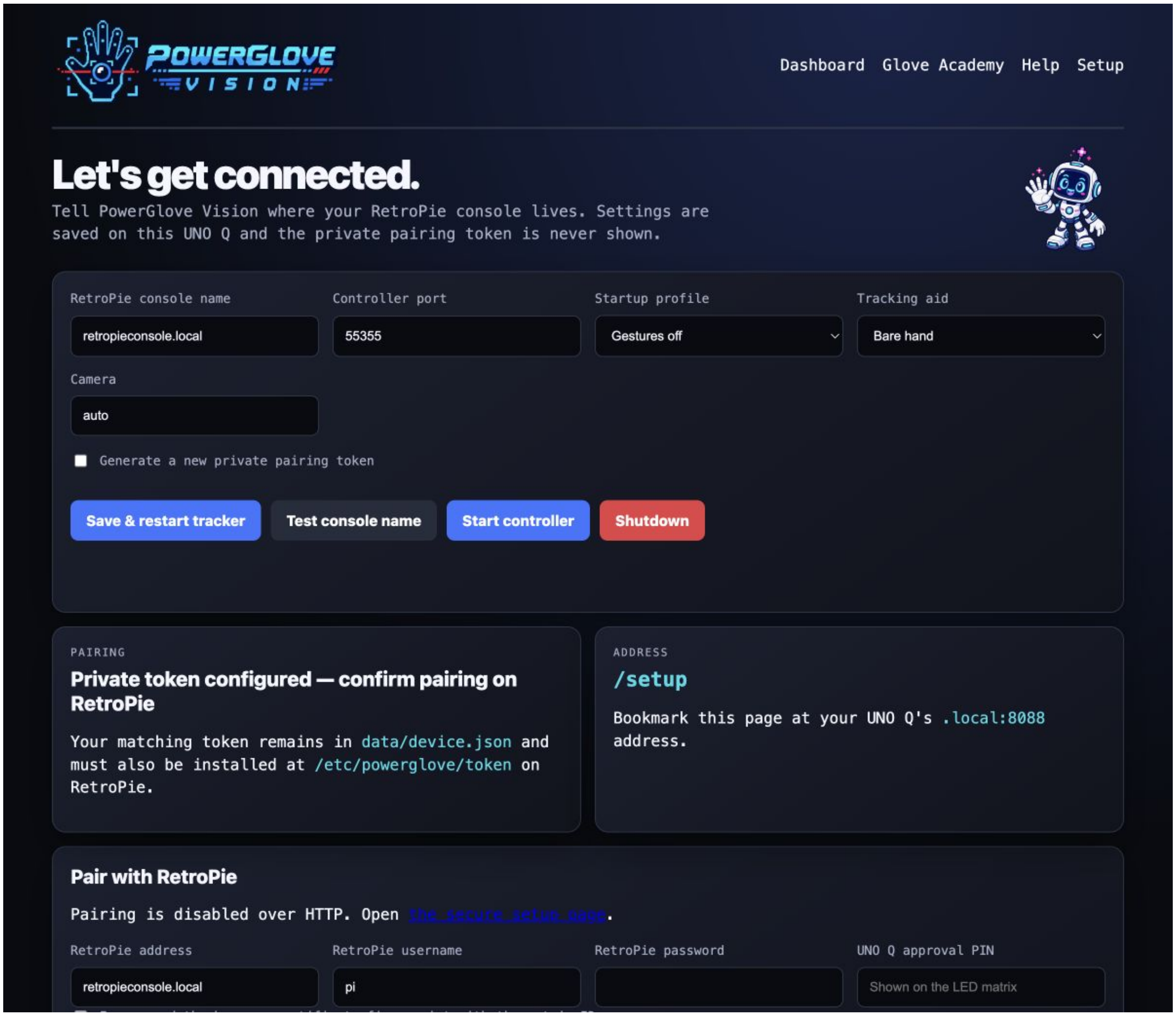
Tune gestures



Tune mode with Pixel Pal guiding the personalization choices

The matrix shows **T** while tuning. Pixel Pal's instruction and primary action sit beside the camera on a wide screen; numerical values and diagnostics are collapsed under **Advanced**.

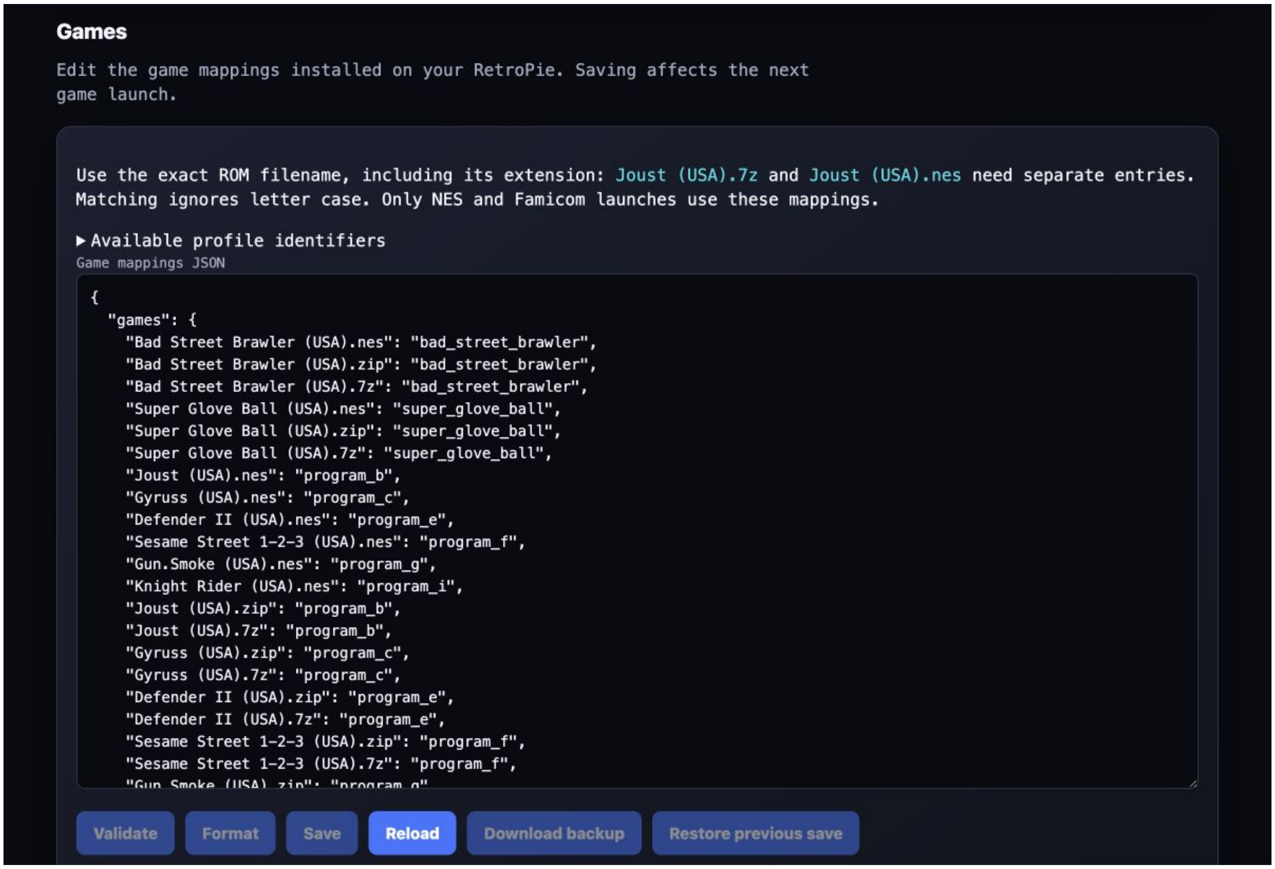
Setup



Setup page for connection settings and pairing

The Setup screenshot shows the HTTP page. Open secure Setup on port 8443 to pair.

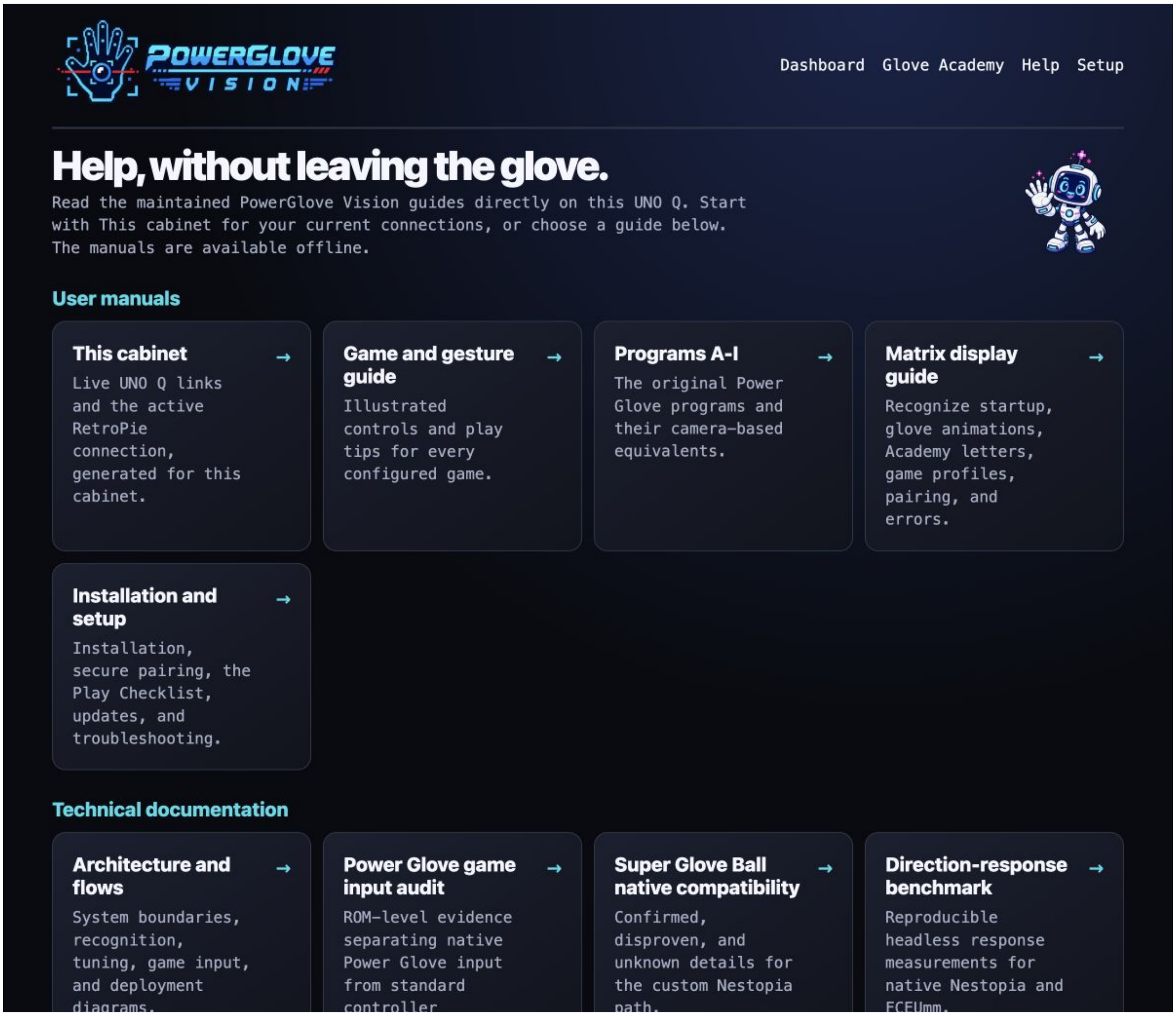
Games within Setup



Games editor below the pairing section

Scroll down Setup to edit mappings. Saving affects the next game launch.

Help



Help page linking to the manuals and printable editions

Choose a profile

Use **Active profile** on Dashboard to choose the controls for your current session. Use **Startup profile** on Setup to choose the profile the app loads when it starts. Available choices are:

- Bad Street Brawler
- Super Glove Ball
- Programs A-I
- Gestures off

Wait for the camera to start

When you select an active profile or open Learn, the app may display **Starting camera and gesture tracking** while it opens the camera and loads the tracker. The elapsed time shows how long initialization has been running. Wait for the live camera view before calibrating.

Gestures off closes the camera. Learn temporarily opens it for practice and suppresses game input. Leaving Learn restores the selected profile.

RetroPie launch hooks select the registered profile when a recognized game starts. A detached monitor waits until RetroArch is running, then renews a bounded game session every two seconds. If you launch an unregistered game or a game for a system other than NES or Famicom, the launch hook selects **Gestures off**. Ending a game, RetroArch stopping, or a session becoming stale also turns gestures off. For a registered game, a one-second post-RetroArch guard pauses controller output so hand movement cannot operate RetroPie's pre-emulator runcommand menu. Output resumes automatically when the guard ends, provided the controller was already armed. A PowerGlove Vision Controller application restart can reconnect on the next renewal while that game remains open. The guard and game session never start a controller that the player explicitly stopped.

Shared recognition and safety gestures

Gesture	See it	Result
Hold a clear V sign steadily for 0.50 seconds		Sends one short Start pulse. Keep a clearly non-V pose visible for 0.30 seconds before Start can trigger again. This prevents an accidental pause while moving or firing.
Briefly show a thumbs-up with the other fingers closed		Sends Select.
Close your hand		Produces the shared closed-hand recognition state; a game profile decides whether it has controller output.
Curl the thumb and ring finger together		Menu guard suppresses movement, A, B, Start, and Select while you reposition your hand. Output returns immediately when the pose ends.

Start and Select poses suppress A/B while they form. Keep your hand near its calibrated center because some profiles can still produce auxiliary output from wrist, depth, or other finger states.

Start with these reusable profiles

Program	Useful for	Controls
A	Pinball and games with two independent actions	Index curl sends A; thumb curl sends Up; wrist roll sends B. Pulling back toggles combined flippers. Ordinary hand movement does not control the D-pad.
D	A reversed-direction challenge	Hand movement sends the opposite direction. Thumb curl sends A; index curl sends B.
H	General NES and Famicom experiments	Hand movement controls the D-pad. Index curl pulses A; thumb curl pulses B. Avoid this profile when a game needs a continuously held action button.

Registered Power Glove games

These assignments select recognition output only; calibration and gesture thresholds remain shared. Except for Super Glove Ball's optional native path, the games use standard NES controller input through FCEUmm.

Game	Profile	Emulator path
Bad Street Brawler	`bad_street_brawler`	FCEUmm
Super Glove Ball	`super_glove_ball`	`lr-nestopia-powerglove` native X/Y, or FCEUmm fallback
Joust	`program_b`	FCEUmm
Gyruss	`program_c`	FCEUmm
Defender II	`program_e`	FCEUmm
Sesame Street 1-2-3	`program_f`	FCEUmm
Gun Smoke	`program_g`	FCEUmm
Knight Rider	`program_i`	FCEUmm

Super Glove Ball: choose native or FCEUmm

Open RetroPie's launch menu while starting Super Glove Ball and choose the emulator for that ROM. RetroPie remembers the per-ROM choice.

- `lr-nestopia-powerglove` is the native path. It uses the shared camera center and safety behavior, but bypasses D-pad thresholds and sends continuous absolute X/Y across the usable camera field. Exact-ROM tests confirm controller detection, native Start, X/Y, signed Z, and open/fist/index packet values. Full-game cabinet play confirms grab/throw, index fire, and fist-plus-forward Power Punch. Movement is playable, with more latency tuning planned. Wrist rotation and remaining unused native packet fields stay neutral.
- `lr-fceumm` remains the complete fallback. It stays in standard joystick mode for the whole session and uses the same responsive movement, finger gestures, and buttons as other FCEUmm games.

Choose FCEUmm again from the same launch menu whenever you want to compare the fallback. A failed or incomplete native setup does not remove it.

Gun Smoke: tested FCEUmm controls

The `program_g` path was tested end to end from the PowerGlove Vision Controller through RetroPie and FCEUmm. Use these controls:

Gesture	Gun Smoke action
Move the whole hand	Walk left, right, up, or down. Returning toward center releases promptly.
Roll wrist left or right	Add left or right movement.
Curl index finger	A: shoot diagonally right.
Push toward camera	B: shoot diagonally left.
Curl index while pushing	A+B: shoot straight ahead.
Curl thumb and ring finger	Menu guard: suppress movement and firing while repositioning.

Use the deliberate V-sign hold for Start or pause. Camera height and distance strongly affect comfort: place the camera first, use a position that leaves the full movement region visible, and then calibrate.

Try a profile in a game

1. Launch an unregistered NES or Famicom game. PowerGlove Vision should show **Gestures off**.
2. Open Dashboard and choose **A: Pinball**, **D: Challenge**, **H: General**, or another profile.
3. Wait for the camera view. Hold your open hand in your comfortable resting position. This is your **neutral position**: the position the app treats as the center for movement.
4. If a direction remains active while your hand is at rest, select **Calibrate** and hold still. Also recalibrate after moving the camera or changing your playing position.
5. Select **Start controller** and test movement, actions, Start, and Select in the game.
6. Select **Stop controller** before adjusting the camera or testing another mapping.

This Dashboard choice is temporary. It does not change the saved startup profile or the game's automatic profile assignment.

Make a game use your chosen profile automatically

Once a profile works well, register the game **on RetroPie**. The launch hook reads `/etc/powerglove/games.json` to choose the profile each time a game starts.

1. Find the game file in your RetroPie ROM folder, usually `~/RetroPie/roms/nes/`. Record its complete filename, including the extension. For example, `/home/pi/RetroPie/roms/nes/My Game (USA).zip` has the filename `My Game (USA).zip`. Use the archive filename when launching an archive, not the filename inside it.
2. Open **Setup -> Games** on the PowerGlove Vision Controller website and select **Download backup**.

3. Edit the loaded JSON in the Games section.
4. Add the filename and your chosen profile inside the existing `games` object. Keep all existing entries, separate entries with commas, and leave no comma after the last entry.
5. Select **Validate**, then **Save**. Wait for verified save confirmation and restart the game. **Restore previous save** reverses the last saved edit.

This example shows the required structure. Replace the example filename with your actual filename and merge the entry into your existing file:

```json
{
  "games": {
    "My Game (USA).zip": "program_h"
  }
}
```

For a manual file edit outside the website, check syntax on RetroPie:

```sh
python3 -m json.tool /etc/powerglove/games.json >/dev/null
```

If the command finishes without reporting an error, the JSON syntax is valid; it does not verify that the filename or profile is correct. Matching ignores letter case but otherwise requires the same filename, including spaces, punctuation, and `.nes`, `.zip`, or `.7z`. Confirm the selected profile on Dashboard after restarting the game.

See the [Gameplay Guide] for game-specific instructions and the [Programs A-I manual] for all reusable mappings.

A **profile queued** launch message means the PowerGlove Vision Controller accepted the request for processing. Confirm the active profile and game name on Dashboard. For timeouts, see [Check a queued profile change]; the PowerGlove Vision Controller must publish UDP `55356`, and the registry must match the exact archive filename.

Tune a gesture

1. Open Learn, show your whole hand, and switch on **Tune gestures**.
2. Tell Pixel Pal whether this is a new hand, a hard gesture, an accidental gesture, or an off-centre play area.
3. Follow one prompt at a time. When tracking has been clear and steady for one second, select **I'm ready** and follow the countdown.
4. Try the preview twice, release it twice, and remain neutral for three seconds.
5. Save the personalization when the guided check passes. Manual values and selective reset are under **Advanced**.

Controller delivery stays paused during tuning. Start it explicitly from Dashboard when ready to play. See [Tune gesture sensitivity] for the recording recipes, neutral calibration, image-quality advice, and shared recognition settings.

Service and configuration reference

Item	Location or name
RetroPie virtual controller	`PowerGlove Vision`
RetroPie pairing token	`/etc/powerglove/token`
RetroPie game registry	`/etc/powerglove/games.json`
RetroPie connection settings	`/etc/powerglove/launcher.json`
Receiver service	`powerglove-receiver.service`
Receiver startup timer	`powerglove-receiver.timer`; starts 45 seconds after boot
PowerGlove Vision Controller shutdown watcher	`powerglove-system-shutdown.path`
PowerGlove Vision Controller shutdown action	`powerglove-system-shutdown.service`; requests a Linux halt
PowerGlove Vision Controller readiness marker	`/home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled`
PowerGlove Vision Controller boot rule that creates the marker	`/etc/tmpfiles.d/powerglove-system-shutdown.conf`; installed from `uno-q/powerglove-system-shutdown.conf`
PowerGlove Vision Controller camera recovery watcher	`powerglove-camera-recovery.path`
PowerGlove Vision Controller camera recovery action	`powerglove-camera-recovery.service`; performs one guarded reset of the last observed parent hub
PowerGlove Vision Controller camera recovery helper	`/usr/local/libexec/powerglove-camera-recovery`; enrolls the single healthy UVC camera on first use
PowerGlove Vision Controller camera recovery allowlist	`/etc/powerglove-camera-recovery.json`; root-owned camera and hub identity/path

The boot rule creates the readiness marker; it does not initiate shutdown or prove that shutdown has completed. The watcher responds to a separate `data/shutdown-request` file created after you confirm **Shutdown** in the browser. Update the rule and its matching service files together using the helper installation command under **Install and deploy over Wi-Fi**.

Verify the helper **on the PowerGlove Vision Controller** without requesting a shutdown:

```
SH
systemctl is-enabled powerglove-system-shutdown.path
systemctl is-active powerglove-system-shutdown.path
systemctl is-enabled powerglove-camera-recovery.path
systemctl is-active powerglove-camera-recovery.path
ls -l /home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled
```

Expect `enabled`, `active`, and an existing marker file. On RetroPie, keep the receiver timer enabled and the receiver service disabled for direct boot activation. The timer starts the service after EmulationStation initializes.

Network ports

Port	Direction	Purpose
TCP 8088	Browser -> PowerGlove Vision Controller	Dashboard, Play, Learn, Help, Setup, status, and camera stream
TCP 8443	Browser -> PowerGlove Vision Controller	Secure Setup and pairing
UDP 55355	PowerGlove Vision Controller -> RetroPie	Controller-state packets
UDP 55356	RetroPie -> PowerGlove Vision Controller	Profile requests and acknowledgements
TCP 55357	PowerGlove Vision Controller -> RetroPie	Temporary one-time-code pairing server

Keep these ports on your trusted local network. Do not expose them to the internet.

Saved calibration and startup

Calibration records your resting hand position, apparent size, and wrist angle in the PowerGlove Vision Controller's `data/calibration.json`. It survives profile changes, Learn sessions, and restarts. Include it in private backups. Recalibrate when your physical setup changes or the resting hand position produces unwanted movement. The app uses 24 clear observations at 70% confidence or better. Returning to the same center, distance, and wrist pose produces a similar reference, although normal camera variation means the saved values will not be exactly equal. Installers preserve this private reference while replacing the shared tested recognition baseline in `config/profiles.json`. The camera overlay's **Right** or **Left** label identifies the hand; its score is confidence in that identification, not confidence in a movement command.

Keep one PowerGlove Vision installation active in App Lab and set it as the default startup app. OpenCV and MediaPipe preload in the background while the website is available. **Gestures off** keeps the camera closed; select an active profile or open Learn to begin capture. An early request waits for preloading to finish. The last explicit Start/Stop choice is restored. An armed controller waits safely for a registered game or intentional manual profile; select **Start controller** when ready to play or **Stop controller** to keep it disarmed.

With preloading complete, the first activation after a tested reboot took 1.21 seconds; actual times vary. For a slow start, inspect the [startup stage logs](). If the camera disappears after reboot, check `lsusb` and `/dev/v4l/by-id/` on the PowerGlove Vision Controller and reconnect the camera or hub if it is missing.

Known limitation: PowerGlove Vision Controller restarts after Shutdown

Stop controller leaves Linux and the website running. **Shutdown** requests a graceful Linux halt. The tested PowerGlove Vision Controller automatically restarts after halt, both with a powered hub and with a direct Mac USB connection. A disappearing website, matrix animation, or fixed waiting period does not confirm that power can safely be removed. See the Installation Guide for the recorded investigation and shutdown guidance.